

SPEC.md — "Visual Campus": AI service that builds a VERIFIED visual profile of a university in ≤ 30 seconds

Instructions to Claude Code (read first, obey throughout):

1. You are implementing this entire project from scratch. **Read this whole file before writing a single line of code.** Before starting each phase (Section 14), re-read the sections it references.
2. **Do not ask the user questions.** When something is unspecified, choose the simplest option that satisfies Section 18 (Definition of Done) and record it in `DECISIONS.md` as one line: `<decision> - <reason>`.
3. **Never invent API parameters.** Every external endpoint is specified in Section 7 with exact parameters. If a real response differs from what Section 7 describes, write a small probe script (`scripts/probe_<service>.py`), run it against the real API, adapt the parser to the real response, and log the discrepancy in `DECISIONS.md`.
4. **Never fabricate data.** No hardcoded photos, no pre-curated lists of images, no fake timings, no fake confidence values, no stock photos. This is a hackathon rule; violating it disqualifies the team. Caching real results is allowed and must be labeled as cached in the UI.
5. Secrets only from environment variables. Never commit `.env`. Always commit `.env.example`.
6. Keep this file in the repo root as `SPEC.md`. Keep `DECISIONS.md` and `EVAL.md` updated.
7. Work strictly phase by phase (Section 14). After each phase: run the tests, run the gate check, commit with a descriptive message. Small, frequent commits.
8. Every failure of an external service must degrade gracefully into a warning, never into a crash or an empty page.

1. Product summary and context

Product: A web service. The user types a university name. Within 30 seconds they receive a structured **visual profile**: photos of the campus, dormitories, classrooms, libraries, laboratories, sport facilities, student life and the city, automatically categorized, de-duplicated, each with a **clickable source link**, a **date** (when available), and an **honest confidence label** explaining why we believe the photo belongs to that university. Plus a short description of the campus with source links, a map with the distance to the city center, and filters.

Context: LOCUS Startup Hackathon 2026, Case 01. The jury will:

- enter several university names, including at least one that is NOT in our demo video;
- measure the time from submitting the query to a useful result;
- inspect the photos, categories, duplicates and source links;
- test behaviour on insufficient data, a misspelled name, and an unavailable source.

Scoring weights: accuracy & relevance 30%, technical implementation & architecture 25%, UI clarity 20%, search speed 15%, scalability & readiness 10%.

Two sentences that define the product philosophy (from the case text):

- Fifteen verified, correctly categorized images are worth more than a hundred random ones.
- A confident output of an unverified image is scored worse than an honest warning.

Therefore: precision over volume, transparency over polish, graceful degradation everywhere.

2. Non-negotiable rules (must be encoded in code and visible in UI)

1. Every displayed photo has a clickable `source_page_url` that opens the page it came from.
2. Every displayed photo has a date field: a real date with its kind (taken / published / uploaded) or the explicit label "date unknown". Never guess a date.
3. Every displayed photo has a verification label — `verified`, `likely`, or `unverified` — computed only from the deterministic rules in Section 10, with machine-readable reason codes rendered as human-readable text.
4. A vision model's judgement alone can never produce `verified`. Web-search results can never reach `verified`.
5. `unverified` photos are hidden by default and shown only when the user toggles "show unverified".

6. If a category has no verified/likely photos, the UI says so explicitly. If the whole profile has fewer than 6 shown photos, the UI shows a "not enough data" banner.
 7. Results served from cache are labeled "cached, built N minutes ago" with a refresh button.
 8. If the time budget is exceeded, the UI says which stage was cut short.
 9. Stock-photo hosts are excluded from web search results (list in Section 7.6).
 10. No manual curation, no per-university hardcoding, no hidden lists. The pipeline must work for any university that exists in Wikidata.
 11. Respect `robots.txt` when scraping official sites. Send an identifying `User-Agent` to Wikimedia APIs (required by their policy).
 12. The 30-second budget is enforced by a hard deadline in code (Section 6), not by hope.
-

3. Tech stack (fixed — do not substitute)

Backend (Python 3.11):

- `fastapi`, `uvicorn[standard]`, `sse-starlette` (SSE responses)
- `httpx` (async HTTP, HTTP/2 off, explicit timeouts), `tenacity` (retries)
- `pydantic v2`, `pydantic-settings`
- `Pillow`, `imagehash` (pHash), `numpy`
- `beautifulsoup4`, `lxml`
- `anthropic` (official SDK, async client)
- `aiosqlite` (cache)
- Dev/test: `pytest`, `pytest-asyncio`, `respx`, `ruff`

Frontend: Vite + React 18 + TypeScript, Tailwind CSS, `react-leaflet` + `leaflet`. No component library. Native `fetch` and `EventSource`.

Packaging/deploy: one Docker image; FastAPI serves `/api/*` and the built frontend (SPA) from the same origin. Node 20 only in the build stage.

Pin all versions (`requirements.txt` with `==`, `package.json` exact versions). Use `ruff` with default rules; every function has type hints.

4. Repository layout

```
visual-campus/
├── SPEC.md                # this file
├── DECISIONS.md           # one line per decision
├── EVAL.md               # results of scripts/eval.py (Phase 7)
├── README.md             # in Russian, Section 17
├── TECHNICAL_NOTES.md    # in Russian, Section 17
├── .env.example
├── .gitignore            # .env, data/, node_modules, dist, __pycache__, .pytest_cache
├── Dockerfile
├── docker-compose.yml    # local dev convenience
├── scripts/
│   ├── eval.py           # runs the university list, prints a markdown table, writes EVAL.md
│   ├── universities.txt  # Section 15.3 list
│   └── probe_*.py        # ad-hoc probes of real APIs (kept, documented)
├── backend/
│   ├── requirements.txt
│   ├── pyproject.toml    # ruff + pytest config
│   └── app/
│       ├── main.py       # FastAPI app factory, static mount, SPA fallback
│       ├── config.py     # Settings (pydantic-settings)
│       ├── models.py     # all pydantic models (Section 5)
│       ├── enums.py      # Category, SourceType, ReasonCode, WarningCode, Verification
│       ├── logging_setup.py
│       ├── http.py       # shared httpx.AsyncClient factory, User-Agent, timeouts
│       └── resolver/
│           ├── wikidata.py # search, entity fetch, parsing, city resolution
│           ├── wikipedia.py # REST summaries
│           └── spelling.py  # LLM spelling normalization fallback
│       ├── sources/
│           └── base.py     # Source protocol: async fetch(ctx) -> list[RawCandidate]
```

```

| | | | commons.py      # category / subcategories / search / single file
| | | | flickr.py
| | | | official_site.py
| | | | google_cse.py    # optional
| | | | registry.py     # enabled sources, priority order, DISABLE_SOURCES
| | | | processing/
| | | | | download.py    # parallel download, validation, thumbnail, hashes
| | | | | dedup.py       # sha1 + pHash union-find
| | | | | textmatch.py   # name/alias normalization and matching
| | | | ai/
| | | | | client.py      # AsyncAnthropic wrapper, retries, JSON parsing
| | | | | vision.py      # batch classification (Section 9)
| | | | | describe.py    # description generation (Section 7.8)
| | | | | prompts.py     # all prompt texts as constants
| | | | scoring.py       # Section 10
| | | | selection.py     # Section 11
| | | | pipeline.py      # orchestration, deadline monitor, event emission
| | | | cache.py         # aiosqlite profile/search cache + thumb dir
| | | | geo.py           # haversine
| | | | api/
| | | | | routes.py      # Section 12
| | | | tests/
| | | | | fixtures/      # real JSON responses saved from probes (sanitized)
| | | | | test_scoring.py
| | | | | test_dedup.py
| | | | | test_textmatch.py
| | | | | test_wikidata_parse.py
| | | | | test_commons_parse.py
| | | | | test_flickr_parse.py
| | | | | test_vision_parse.py
| | | | | test_pipeline_degraded.py
| | | | frontend/
| | | | | package.json
| | | | | vite.config.ts    # dev proxy /api -> http://localhost:8000
| | | | | index.html
| | | | | src/
| | | | | | main.tsx
| | | | | | App.tsx        # routes: "/" and "/u/:qid"
| | | | | | api.ts         # fetch + EventSource wrappers, typed
| | | | | | types.ts       # mirrors Section 5 (generated by hand, keep in sync)
| | | | | | i18n.ts        # ru (default) + en strings, reason/warning code mappings
| | | | | | pages/SearchPage.tsx
| | | | | | pages/ProfilePage.tsx
| | | | | | components/    # Section 13

```

5. Data model (backend pydantic models; frontend `types.ts` mirrors them exactly)

```

# enums.py
class Category(str, Enum):
    campus = "campus"; dormitory = "dormitory"; classroom = "classroom"; library = "library"
    lab = "lab"; sport = "sport"; student_life = "student_life"; city = "city"; other = "other"

class SourceType(str, Enum):
    wikidata_p18 = "wikidata_p18"; official_site = "official_site"; commons_category = "commons_category"
    flickr_geo = "flickr_geo"; commons_search = "commons_search"; web_search = "web_search"
    city_commons = "city_commons"

# Source priority (highest first) - used for dedup representative choice and candidate caps:
SOURCE_PRIORITY = [wikidata_p18, official_site, commons_category, flickr_geo, commons_search, web_search, city_commons]

class Verification(str, Enum):
    verified = "verified"; likely = "likely"; unverified = "unverified"

class ReasonCode(str, Enum):
    wikidata_main_image = "wikidata_main_image"
    official_site_source = "official_site_source"
    commons_category_source = "commons_category_source"
    commons_search_source = "commons_search_source"
    flickr_geo_source = "flickr_geo_source"
    web_search_source = "web_search_source"
    city_category_source = "city_category_source"

```

```

name_in_metadata = "name_in_metadata"      # name/alias found in title, description, filename, alt or page title
name_on_sign = "name_on_sign"              # vision model read the name on a sign/banner in the image
geo_within_300m = "geo_within_300m"
geo_within_1km = "geo_within_1km"
vision_consistent = "vision_consistent"    # is_photo, plausibly related, category != other
vision_unavailable = "vision_unavailable"    # model failed / no key
vision_timeout = "vision_timeout"           # skipped by deadline monitor
category_heuristic = "category_heuristic"   # category from filename/caption keywords
cached_result = "cached_result"

class WarningCode(str, Enum):
    low_data = "low_data"; no_coordinates = "no_coordinates"; no_commons_category = "no_commons_category"
    no_official_site = "no_official_site"; source_unavailable = "source_unavailable" # detail = source name
    vision_unavailable = "vision_unavailable"; time_budget_exceeded = "time_budget_exceeded"
    served_from_cache = "served_from_cache"; ambiguous_name = "ambiguous_name"
    missing_category = "missing_category" # detail = category name
    queued = "queued"

# models.py
class Coordinates(BaseModel): lat: float; lon: float

class Candidate(BaseModel):
    qid: str; label: str; description: str | None = None
    country: str | None = None; city: str | None = None

class SearchResponse(BaseModel):
    query: str; candidates: list[Candidate]
    suggestions_used: bool = False; corrected_query: str | None = None

class Place(BaseModel):
    qid: str | None = None; name: str; coords: Coordinates | None = None
    wikipedia_url: str | None = None; commons_category: str | None = None

class UniversityHeader(BaseModel):
    qid: str; name: str; local_name: str | None = None; aliases: list[str] = []
    country: str | None = None; city: Place | None = None; coords: Coordinates | None = None
    official_website: str | None = None; wikipedia_url: str | None = None
    commons_category: str | None = None; logo_url: str | None = None
    distance_to_city_center_km: float | None = None
    cached: bool = False; cached_at: datetime | None = None

class SourceLink(BaseModel): title: str; url: str

class Description(BaseModel):
    text_ru: str; text_en: str
    sources: list[SourceLink]
    basis: Literal["wikipedia_and_site", "wikipedia_only", "site_only", "insufficient"]

class Photo(BaseModel):
    id: str # first 16 hex chars of sha1(source_page_url + "|" + image_url)
    thumb_url: str # "/api/thumb/{id}.jpg" — always served by our backend
    image_url: str # original image URL (for the lightbox link, not for <img>)
    source_page_url: str # clickable source
    source_type: SourceType
    source_label: str # "Wikimedia Commons" | "Flickr" | "<hostname of official site>" | "<displayLink host>"
    title: str | None = None; author: str | None = None; license: str | None = None
    date: str | None = None # "YYYY-MM-DD" (time dropped) or None
    date_kind: Literal["taken", "published", "uploaded", "unknown"] = "unknown"
    category: Category
    category_source: Literal["vision", "heuristic", "source_hint"]
    width: int | None = None; height: int | None = None
    confidence: float # 0.0-1.0, rounded to 2 decimals
    verification: Verification
    reasons: list[ReasonCode]
    geo: Coordinates | None = None; distance_m: float | None = None
    visible_text: str | None = None; vision_reason: str | None = None

class Stats(BaseModel):
    found: int; duplicates_removed: int; irrelevant_removed: int
    shown: int; hidden_unverified: int
    per_source: dict[str, int] # raw candidates per source
    timings_ms: dict[str, int] # resolve, collect, download, dedup, vision, describe, total
    total_ms: int

```

```
class Warning(BaseModel): code: WarningCode; detail: str | None = None

class Profile(BaseModel):
    header: UniversityHeader; description: Description | None
    photos: list[Photo]          # shown photos (verified + likely) AND up to 15 unverified (hidden by UI toggle)
    stats: Stats; warnings: list[Warning]; generated_at: datetime
```

Internal (not exposed): `RawCandidate` (`source_type`, `image_url`, `source_page_url`, `title`, `author`, `license`, `date`, `date_kind`, `geo`, `page_title`, `filename`, `width`, `height`, `source_hint_category`) and `ProcessedImage` (`RawCandidate` + `bytes sha1`, `phash`, `thumb path`, `base64 512px`, `width`, `height`).

6. Pipeline — stages, order, time budgets

Global: `HARD_DEADLINE_S = 27` (env). `start = monotonic()` at request. Every stage uses `asyncio.wait_for` with the budgets below; the pipeline never raises to the client — it emits `warning` events and continues.

Stage	What	Target	Hard limit	Emits
A. resolve	Wikidata entity (or search if no qid), city chain, Wikipedia summaries (en, ru) in parallel	2.0 s	5 s	header
B. collect	All enabled sources concurrently (Section 7): <code>wikidata_pl8</code> , <code>commons_category</code> (+subcats), <code>commons_search</code> , <code>flickr_geo</code> , <code>official_site</code> , <code>city_commons</code>	6 s	8 s per source	warning per failed source
B2. web_search	Only if <code>GOOGLE_CSE_*</code> set AND non-city candidates after B < 12 AND elapsed < 12 s	3 s	5 s	—
C. normalize	URL-dedupe across sources, apply caps (Section 6.1)	~0	—	—
D. download	Parallel thumbnail download + validation + pHash + 512px JPEG (Section 8)	4 s	6 s per image, 8 s stage	—
E. dedup	sha1 exact + pHash near-duplicate grouping (Section 8)	~0	—	—
F. vision	Batched classification (Section 9), ordered by source priority	10 s	20 s per batch	photos chunk after each scored batch
G. score	Section 10 per photo, right after its batch	~0	—	(inside photos)
H. describe	Runs in parallel with F (Section 7.8)	6 s	12 s	description
I. finalize	Selection caps (Section 11), stats, warnings, cache write	~0	—	stats, done

Deadline monitor: before starting each vision batch check `remaining = HARD_DEADLINE_S - elapsed`. If `remaining < 4` → do not start; cancel pending batches; every unclassified image gets `category` via heuristic (Section 11.4), reasons `+vision_timeout`, scoring without vision (so at most likely), and warning `time_budget_exceeded` with detail `"vision: X of Y images not classified"`. Emit them, finalize, and finish before 27 s wall time.

6.1 Caps after normalization: keep at most `MAX_CANDIDATES = 120` non-city candidates, taken in `SOURCE_PRIORITY` order (all of a higher-priority source before the next), preserving API order within a source. City candidates: separate cap 20.

6.2 Concurrency guard: at most 3 pipelines run concurrently (`asyncio.Semaphore`). A 4th request waits and receives `warning: queued`.

6.3 Cached path: if a fresh cached profile exists (Section 12.4) and `refresh != 1`, emit header (with `cached=true`), `description`, one `photos` chunk with everything, `stats`, `done` immediately; warning `served_from_cache`.

7. External integrations — exact endpoints, parameters, parsing, filters

Shared HTTP rules (`http.py`): one `httpx.AsyncClient(follow_redirects=True, timeout=httpx.Timeout(8.0, connect=4.0), limits=httpx.Limits(max_connections=40, max_keepalive_connections=20))`. Default headers: `User-Agent: VisualCampus/1.0 (https://github.com/<team>/visual-campus; {CONTACT_EMAIL})` and `Accept: application/json` for API calls. Retry policy (tenacity): up to 2 retries on connection errors, 429, 502, 503, 504 with exponential backoff 0.5 → 2 s; never retry on 4xx other than 429. Log each external call with service name, URL (without keys), status, and elapsed ms.

7.1 Wikidata (resolver)

Base URL: `https://www.wikidata.org/w/api.php`. Always add `format=json`. Timeout 5 s.

Search (Stage A when no qid): `action=wbsearchentities&search={q}&language={lang}&uselang=en&type=item&limit=10` for `lang` in `["en", "ru", "kk"]` executed in parallel; merge by `id`, keep the best (lowest) rank across languages. Response: `search[]` with `id`, `label`, `description`, `match`.

Entity fetch: `action=wbgetentities&ids=`

`{id1|id2|...}&props=claims|labels|descriptions|aliases|sitelinks&languages=en|ru|kk&sitefilter=enwiki|ruwiki|kkwiki` (max 50 ids per call).

Response: `entities[qid]`.

University filter — keep a candidate if (a) OR (b):

- (a) any `claims.P31[*].mainsnak.datavalue.value.id` is in `UNIVERSITY_QIDS = {"Q3918", "Q38723", "Q875538", "Q902104", "Q15936437", "Q189004", "Q1371037", "Q4671277", "Q2385804"}` (labels: university, higher education institution, public university, private university, research university, college, institute of technology, academic institution, educational institution). In Phase 1, verify these labels with one `wbgetentities&ids=...&props=labels&languages=en` call and remove any QID whose label does not match; record the check in `DECISIONS.md`.
- (b) the `en` or `ru` description matches (case-insensitive regex): `university|universit|college|institute|polytechnic|academy|school of|conservator|университет|институт|академия|колледж|вуз|консерватор|университети|институти`

Rank: search order; move exact case-insensitive label/alias matches to the top. Return up to 8 Candidate s. Country/city labels for candidates: collect all `P17` and `P131` QIDs of kept candidates, fetch their labels in one `wbgetentities&props=labels&languages=en|ru` call.

Fallbacks when zero candidates survive:

1. Wikipedia opensearch: `https://en.wikipedia.org/w/api.php?action=opensearch&search={q}&limit=5&namespace=0&format=json` and the same on `ru.wikipedia.org`. Map titles → QIDs: `https://(lang).wikipedia.org/w/api.php?action=query&prop=pageprops&ppprop=wikibase_item&titles={t1|t2|...}&format=json` → `query.pages[*].pageprops.wikibase_item`. Then entity fetch + filter as above.
2. LLM spelling normalization (Section 7.8.b): get up to 3 probable official English names → re-run `wbsearchentities` (en) for each → filter. If this produced the results, set `suggestions_used=true`, `corrected_query=<first name that worked>`.
3. Still nothing → `candidates=[]` (UI shows the not-found state).

Entity parsing → UniversityReader (from `entities[qid]`); skip any claim whose `mainsnak.snaktype != "value"`; prefer claims with `rank == "preferred"`, else first):

- `name`: `labels.en.value` → else `ru` → else `kk` → else first label.
- `local_name`: `labels.ru.value` or `labels.kk.value` if it differs from `name`.
- `aliases`: `aliases.{en,ru,kk}[*].value` + `P1448` (official name, `value.text`) + `P1813` (short name, `value.text`); dedupe; drop entries shorter than 3 characters.
- `coords`: `P625` → `value.latitude`, `value.longitude`.
- `official_website`: `P856` → `value` (string).
- `commons_category`: `P373` → `value` (string).
- `main image filename`: `P18` → `value` (string, e.g. "MIT Dome.jpg"); `logo filename`: `P154`.
- `country`: `P17` → QID → label (en).
- `wikipedia_url`: `sitelinks.enwiki.title` → `https://en.wikipedia.org/wiki/{title with spaces→_}`; if no enwiki, use ruwiki.
- **City resolution:** `cur = P131[0]`; repeat up to 4 hops: fetch `cur` (`props=claims|labels|sitelinks&languages=en|ru&sitefilter=enwiki|ruwiki`); if its `P31` intersects `CITY_QIDS = {"Q515", "Q1549591", "Q3957", "Q5119", "Q200250", "Q7930989", "Q1093829"}` (city, big city, town, capital city, metropolis, city/town, city of the United States — verify labels in Phase 1 as above) → this is the city; else `cur = its P131[0]`. If no hop matched: use `P159` (headquarters location) if it has `P625`; else the first hop entity that has both `P625` and an `enwiki` / `ruwiki` sitelink. City `Place`: name (en label, ru fallback), `coords` `P625`, `commons_category` `P373`, `wikipedia_url` from `sitelinks`.
- `distance_to_city_center_km`: `haversine(university coords, city coords)`, 1 decimal; `None` if either missing.

7.2 Wikipedia REST summary

GET `https://(lang).wikipedia.org/api/rest_v1/page/summary/{title}` with title URL-encoded, spaces as `_`. Fields used: `extract` (plain text), `content_urls.desktop.page`. Timeout 4 s. Fetch `en` and `ru` when `sitelinks` exist. Used only as input to Section 7.8 and as `Description.sources`.

7.3 Wikimedia Commons

Base: `https://commons.wikimedia.org/w/api.php`, `format=json`, `timeout 8 s`. Common `prop` block for every file query:

`prop=imageinfo&iiprop=url|extmetadata|size|mime|timestamp&iurlwidth=640&iextmetadatafilter=DateTimeOriginal|LicenseShortName|Artist|ImageDescr`

- (a) **Category files:** `action=query&generator=categorymembers&gcmtitle=Category:{commons_category}&gcmtype=file&gclimit=50<prop block>`. Follow `continue.gcmcontinue` at most once (max 100 files).
- (b) **Subcategories:** `action=query&list=categorymembers&cmttitle=Category:{commons_category}&cmttype=subcat&cmlimit=50`. Choose up to 8

subcategories whose title matches (case-insensitive)

building|campus|librar|dormitor|hostel|residen|student|sport|stadium|laborator|facult|interior|hall|auditorium|здани|кампус|общежит|библиотек ; if fewer than 3 match, take the first 8 in API order. For each chosen subcategory run (a) with `gcmlimit=30`, no continuation. Subcategory titles that match `librar` → `source_hint_category=library`, `dormitor|hostel|residen` → `dormitory`, `sport|stadium` → `sport`, `laborator` → `lab`, `student` → `student_life` (hint only; vision has precedence).

- **(c) Search (always run):** `action=query&generator=search&gsrsearch={name}&gsrnamespace=6&gsrlimit=40&<prop block>`. Run with `name = English` label; additionally with `local_name` if it exists (2 calls max).
- **(d) Single file (P18, logo):** `action=query&titles=File:{filename}&<prop block>`.
- **(e) City category:** (a) on `city.common`s_category with `gcmlimit=30`, `source_type=city_commons`.

Parsing each `query.pages[*]: title ( "File:..." )`, `imageinfo[0]: descriptionurl` → `source_page_url` (fallback

`https://commons.wikimedia.org/wiki/{urlencoded title}`); `thumburl` → `image_url` to download (fallback url); `width`, `height`, `mime`; `extmetadata.DateTimeOriginal.value` → parse with regex `^(\d{4})[-:/](\d{2})[-:/](\d{2})` → `date="YYYY-MM-DD"`, `date_kind="taken"`; if absent/unparseable and `timestamp` exists → `date=timestamp[:10]`, `date_kind="uploaded"`; `LicenseShortName.value` → `license`; `Artist.value` → strip HTML with BeautifulSoup → `author` (≤ 80 chars); `ImageDescription.value` → strip HTML → `title` (≤ 300 chars); `filename` = title without `File:`.

Filters (before download): `mime` ∈ {`image/jpeg`, `image/png`, `image/webp`}; `width` ≥ 400 and `height` ≥ 300; `filename` NOT matching (case-insensitive)

`logo|emblem|coat[_]of[_]arms|seal|crest|flag|bmap|b|plan|scheme|diagram|chart|icon|screenshot|poster|document|certificate|diploma|portrait|passport|table|graph|\.svg$`

Limit concurrent downloads from `upload.wikimedia.org` to 8 (a dedicated semaphore) to avoid 429.

7.4 Flickr

Requires `FLICKR_API_KEY`; if missing, skip the source and emit `warning source_unavailable` with detail `flickr` (no API key). **Base:**

`https://www.flickr.com/services/rest/`, **timeout** 8 s, **always** `format=json&nojsoncallback=1`.

- **(a) Geo search:** `method=flickr.photos.search&api_key={KEY}&lat={lat}&lon={lon}&radius=1&radius_units=km&has_geo=1&min_taken_date=2008-01-01&content_type=1&media=photos&safe_search=1&sort=relevance&per_page=60&page=1&extras=date_taken,license,geo,url_m,url_z,url_l,owner_name,o_d` (`min_taken_date` is mandatory: Flickr rejects geo queries without a "limiting agent".) Skipped if no coords.
- **(b) Text search near campus:** same as (a) but `text={english label}`, `radius=5`, `per_page=40`.
- **License names:** at startup call `method=flickr.photos.licenses.getInfo&api_key={KEY}` once, cache {`id`: `name`}; if it fails use the hardcoded map {`0`: "All Rights Reserved", `1`: "CC BY-NC-SA 2.0", `2`: "CC BY-NC 2.0", `3`: "CC BY-NC-ND 2.0", `4`: "CC BY 2.0", `5`: "CC BY-SA 2.0", `6`: "CC BY-ND 2.0", `7`: "No known copyright restrictions", `8`: "US Government Work", `9`: "CC0", `10`: "Public Domain Mark"}.

Parsing `photos.photo[*]: id`, `owner`, `title`, `ownername` → `author`, `datetaken ( "YYYY-MM-DD HH:MM:SS" )` → `date=[:10]`, `date_kind="taken"`;

`license` → `name`; `latitude` / `longitude` (strings; 0 / "0" means none); `url_z` (640 px) else `url_m` (500 px) else skip; `width_z` / `height_z` (or `width_m` / `height_m`); `description._content` (≤ 300 chars); `tags` (space-separated). `source_page_url` = `https://www.flickr.com/photos/{owner}/{id}/`, `source_label="Flickr"`. Compute `distance_m` to campus coords.

Filters: skip if no usable URL; skip if title/tags/description match the Section 7.3 exclusion regex.

7.5 Official website (P856)

Skipped with `warning no_official_site` if `P856` missing. Budget: 8 s for the whole source; per page 6 s; max 7 pages (homepage + 6); concurrency 4.

Headers: `User-Agent: Mozilla/5.0 (compatible; VisualCampusBot/1.0; +mailto:{CONTACT_EMAIL})`, `Accept: text/html, */*`. Follow redirects. Ignore TLS errors? **No** — treat as failure. `robots.txt`: `fetch {scheme}://{host}/robots.txt` (timeout 3 s) into `urllib.robotparser.RobotFileParser`; skip any URL it disallows for `*`; on fetch failure assume allowed.

Steps:

1. Fetch homepage. On failure → `warning source_unavailable` **detail** `official_site` and stop.
2. Parse with BeautifulSoup(`html`, "`lxml`"). Collect image URLs from: `meta[property="og:image"]`, `meta[name="twitter:image"]`, `<img>` attributes `src`, `data-src`, `data-lazy-src`, `data-original`, and the largest candidate in `srcset`; `<source srcset>`; `inline style` attributes via regex `url\([\'"]?\)[^\'"]*\)`. Resolve with `urljoin(page_url, u)`. Keep `http(s)` only.
3. Collect internal links (`<a href>`) on the same registrable domain (compare last two labels of the host, e.g. `nu.edu.kz` ↔ `admissions.nu.edu.kz`) whose `href` or anchor text matches (case-insensitive) `campus|dorm|hostel|residen|housing|accommodation|library|student.?life|sport|gym|about|tour|gallery|photo|facilit|кампус|общежит|библиотек|студенческ|спорт|галере|жатакхана|китапхана`; take the first 6 unique; fetch in parallel; repeat step 2 on each.
4. Per image: `source_page_url` = the page where it was found; `source_label` = host of official site; `title` = `alt` text or `og:title` / `<title>` of the page; `date`: `meta[property="article:published_time"]` or `meta[name="date"]` (parse `YYYY-MM-DD`) → `date_kind="published"`; else `Last-Modified` response header → `published`; else unknown.

5. Filters: URL not matching

`logo|icon|sprite|avatar|flag|badge|button|arrow|pixel|track|banner|adv|favicon|loader|spinner|placeholder|blank|.svg|.gif`; after download: min side ≥ 300 px, bytes ≥ 15 KB, aspect ratio between 0.4 and 2.6. Cap 40 candidates (homepage `og:image` first, then in discovery order).

JS-only sites yield few images; that is acceptable — degrade silently (only the homepage failure is a warning).

7.6 Google Custom Search (optional booster)

Only if `GOOGLE_CSE_KEY` and `GOOGLE_CSE_CX` are set. Conditions in Section 6 (Stage B2). Max 2 queries per profile: "{label} campus" and "{label} student life". GET `https://www.googleapis.com/customsearch/v1?key={KEY}&cx={CX}&q={query}&searchType=image&num=10&safe=active&imgSize=large&imgType=photo` Parse `items[*]: link → image_url; image.contextLink → source_page_url; title; displayLink → source_label; image.width/height. date_kind="unknown"`. Drop results whose `displayLink` matches `shutterstock|gettyimages|iStockphoto|alamy|depositphotos|dreamstime|123rf|stock.adobe|adobestock|unsplash|pexels|pixabay|freepik|canva|vecteezy` Base weight 0.15 (Section 10). Quota is 100 queries/day — never call it on the cached path.

7.7 Anthropic — vision classification

See Section 9 for the prompt and batching. SDK: `from anthropic import AsyncAnthropic; client created once with api_key=settings.anthropic_api_key`. Model from env `VISION_MODEL`, default `claude-haiku-4-5-20251001`. In Phase 3 verify the model id against the Anthropic docs models page; if it has changed, update the default in `config.py` and `.env.example`. At startup, if `ANTHROPIC_API_KEY` is set, run one tiny text call (`max_tokens=5`) and log success/failure; failure → log a loud warning, vision/description become unavailable, app still serves.

Image content block format (base64 JPEG, ≤ 512 px long side):

```
{ "type": "image", "source": { "type": "base64", "media_type": "image/jpeg", "data": "<base64>" }
```

7.8 Anthropic — text tasks (same client, model `TEXT_MODEL`, default = `VISION_MODEL`)

(a) Description. Input: `name`, `local_name`, `city`, `country`, Wikipedia `extract` (en, ru; each truncated to 1500 chars), official site `<title>` + `meta[name=description]` if fetched, list of available source URLs. `max_tokens=700`, `temperature=0`. Prompt (constant in `prompts.py`):

```
You write a short, factual campus description for prospective students, based ONLY on the provided source texts. Do not add facts that are not in the sources. Return ONLY a JSON object: { "text_ru": "<3-5 sentences in Russian>", "text_en": "<3-5 sentences in English>", "basis": "wikipedia_and_site_cover", when available: where the campus is (city, district), what the campus is like (buildings, size, notable facilities: libraries, dorms, etc.) }
```

`Description.sources` = Wikipedia page URL(s) + official site URL that were actually provided as input. If the call fails → fallback: `text_en` = first 3 sentences of the en extract (or ru), `text_ru` = first 3 sentences of the ru extract (or the en text), `basis` per availability; if no extract at all → `basis="insufficient"`, `text_ru="Недостаточно данных для описания кампуса из открытых источников."`, `text_en="Not enough open-source data to describe the campus."`.

(b) Spelling normalization (resolver fallback only). `max_tokens=200`, `temperature=0`:

```
The user typed a university name that was not found: "{q}". Return ONLY a JSON array of up to 3 strings with the most probable official English names.
```

8. Image processing (Stage D/E)

- Download concurrency `DOWNLOAD_CONCURRENCY=16` (plus the 8-limit for `upload.wikimedia.org`). Per image: timeout 6 s; stream the body; abort if `Content-Length` or accumulated bytes > 4 MB; require `Content-Type` starting with `image/` (if the header is missing, try to decode anyway).
- Decode in a thread (`asyncio.to_thread`): `Image.MAX_IMAGE_PIXELS = 40_000_000`; `Image.open(BytesIO(b))`; `img.load()`; `img = ImageOps.exif_transpose(img).convert("RGB")`. Reject if `min(w, h) < 250` or aspect ratio > 3.5 or < 0.28.
- Compute `sh1(bytes)`, `phash = imagehash.phash(img)` (default `hash_size=8`).
- Thumbnail: resize so the long side is 512 px (`Image.LANCZOS`), save JPEG quality 82 to `{CACHE_DIR}/thumbs/{photo_id}.jpg`; keep its base64 in memory for the vision batch.
- Dedup (union-find): (1) identical `sh1` → same group; (2) `phash_a - phash_b <= 6` → same group; (3) `7..12` → near-duplicate, same group. Compare all pairs (≤ 120 items → trivial). Representative = highest `SOURCE_PRIORITY`, then larger `width*height`. Non-representatives are dropped; count → `stats.duplicates_removed`.
- Also drop any image whose `sh1 / phash` (≤ 8) equals the university logo (`P154`) when available.

9. Vision classification (Stage F)

Batching: `VISION_BATCH_SIZE=8` images per request, `VISION_CONCURRENCY=4` requests in flight, per-batch timeout 20 s. Batches ordered by

SOURCE_PRIORITY of their images (best sources first) so early photos chunks are the strongest. City images are batched separately with context.target="city".

Request: messages.create(model=VISION_MODEL, max_tokens=900 + 130*N, temperature=0, system=VISION_SYSTEM, messages=[{"role": "user", "content": [{text: header}, {text: "IMAGE 1"}, {image 1}, {text: "IMAGE 2"}, {image 2}, ..., {text: footer}]}]).

VISION_SYSTEM (exact):

You are an image-classification component inside a pipeline that builds a verified visual profile of a university for prospective student.

Header text (filled per batch):

University: {name} (aliases: {aliases joined by "; "}). City: {city}, {country}.

Classify each of the N={N} images below. For each image return an object:

```
{"index": <1..N>,
  "is_photo": <true|false>,    // false for logos, emblems, maps, floor plans, diagrams, 3D renders/visualizations, screenshots, documents,
  "category": "campus"|"dormitory"|"classroom"|"library"|"lab"|"sport"|"student_life"|"city"|"other",
  "category_confidence": <0.0-1.0>,
  "plausibly_university_related": <true|false>,    // the scene could plausibly be a university campus/facility/student activity, or the name
  "name_on_sign": <true|false>,    // true ONLY if readable text in the image clearly contains the university name or an alias (any language)
  "visible_text": <string up to 80 chars, or null>,
  "people_prominent": <true|false>,    // a person or face is the main subject
  "quality_ok": <true|false>,    // false if blurry, tiny, heavily watermarked, or dominated by overlaid text/advertising
  "reason": <string, max 12 words>}
```

Category definitions:

- campus: exteriors of university buildings, main entrance/gates, quad, courtyard, aerial view, academic buildings, general grounds.
- dormitory: student residence/hostel exterior or interior (rooms, corridors, shared kitchens, lobbies).
- classroom: lecture halls, auditoriums, seminar rooms, classrooms, lectures in progress.
- library: reading rooms, book stacks, library halls, library interiors; a building exterior only if it is clearly a library.
- lab: laboratories, research facilities, workshops with equipment, computer labs, clean rooms, simulation rooms.
- sport: stadiums, gyms, pools, courts, fields, students doing sport.
- student_life: events, ceremonies, graduation, clubs, canteens, festivals, concerts, fairs, groups of students socializing.
- city: cityscapes, streets, landmarks, panoramas, transport, parks that are NOT on campus.
- other: anything else, and everything with is_photo=false.

When two categories fit, prefer in this order: dormitory > library > lab > classroom > sport > student_life > campus > city.

Footer text (exact):

Return ONLY a JSON array with exactly N objects, ordered by index 1..N. No markdown, no comments, no text outside the array.

Parsing: text = "".join(block.text for block in resp.content if block.type == "text") ; strip ```json / ``` fences; json.loads ; validate with a pydantic model VisionItem ; require len == N and indexes 1..N . On failure: retry once with the same content plus a final text block "Your previous output was not a valid JSON array of N objects. Output only the JSON array." . On second failure, or on API error after retries (429/5xx: tenacity 2 retries, backoff 1–4 s), mark every image in the batch vision_unavailable (no crash, no drop).

Post-processing per image (before scoring):

- If is_photo == false OR people_prominent == true OR quality_ok == false → **remove**, stats.irrelevant_removed += 1 .
- If plausibly_university_related == false and category != "city" → **remove**, irrelevant_removed += 1 .
- If category == "other" → **remove**, irrelevant_removed += 1 .
- For source_type == city_commons : force category = city (keep the is_photo removal rule only).
- category_source = "vision" .

10. Scoring and verification (Stage G) — deterministic, no exceptions

confidence = min(1.0, base + Σ bonuses) ; round to 2 decimals. Append the matching ReasonCode for every term that applies.

Base by source_type :

source_type	base	reason
wikidata_p18	0.60	wikidata_main_image
official_site	0.55	official_site_source
commons_category	0.55	commons_category_source

city_commons	0.55	city_category_source
flickr_geo	0.40	flickr_geo_source
commons_search	0.25	commons_search_source
web_search	0.15	web_search_source

Bonuses:

- +0.20 `name_in_metadata` — the normalized name or any alias (length ≥ 4) occurs in normalized `title`, `description`, `filename`, `alt`, or `page_title`. Normalization (`textmatch.py`): NFKD $\rightarrow$ drop combining marks $\rightarrow$ casefold $\rightarrow$ replace `[\^\w\s]` with space $\rightarrow$ collapse whitespace. Not applied to `official_site` and `city_commons` (already implied).
- +0.20 `geo_within_300m` if `distance_m` < 300 ; else +0.10 `geo_within_1km` if `distance_m` < 1000 (only sources with geo — Flickr).
- +0.15 `vision_consistent` — vision succeeded, `is_photo`, `plausibly_university_related`, `category` != `other`.
- +0.15 `name_on_sign` — vision returned `name_on_sign` == `true`.

Labels: `confidence` $\geq 0.70 \rightarrow$ `verified`; `0.40` $\leq$ `confidence` < `0.70` $\rightarrow$ `likely`; < `0.40` $\rightarrow$ `unverified`.

Hard rules (applied after thresholds):

- If vision was unavailable or timed out for the image $\rightarrow$ label at most `likely`; add `vision_unavailable` / `vision_timeout`.
- `web_search` images $\rightarrow$ label at most `likely` (by construction `max` = `0.15+0.20+0.15+0.15` = `0.65`; enforce anyway).
- `city_commons` images are verified against the `city`, and the UI reason text must say so.

Worked examples (must be unit-tested): `commons_category` + name in `filename` + vision $\rightarrow$ `0.90` verified; `commons_category` + vision only $\rightarrow$ `0.70` verified; `commons_search` + `name_in_metadata` + vision $\rightarrow$ `0.60` likely; `commons_search` + `name_in_metadata` + vision + `name_on_sign` $\rightarrow$ `0.75` verified; flickr 200 m + vision $\rightarrow$ `0.75` verified; flickr 800 m + vision $\rightarrow$ `0.65` likely; `official_site` + vision $\rightarrow$ `0.70` verified; `official_site` without vision $\rightarrow$ `0.55` likely; `web_search` + name + vision + sign $\rightarrow$ `0.65` likely.

11. Selection, caps, categories, warnings (Stage I)

1. Split photos into `shown` (verified + likely) and `hidden` (unverified).
2. Sort `shown` by (verification: verified first, confidence desc, SOURCE_PRIORITY). Per-category cap 12; `city` cap 6; total shown cap 45. Keep the cut ones as unverified? **No** — cut ones are simply dropped and counted in `stats.found` only.
3. Include up to 15 `hidden` in `Profile.photos` (highest confidence first) — the UI hides them behind the toggle. `stats.hidden_unverified` = `len(hidden)`.
4. **Heuristic category** (used when vision is unavailable/timed out): match `filename` + `title` + `page_title` (normalized) against, in order: `dormitory` `dorm|hostel|residen|общежит|жатакхана` ; `library` `librar|библиотек|китапхана` ; `lab` `lab\b|laborator|лаборатор` ; `classroom` `lecture|auditorium|classroom|аудитор|лекци` ; `sport` `sport|stadium|gym|pool|спорт|стадион|бассейн` ; `student_life` `student|graduat|ceremon|festival|студент|выпуск` ; `city sources` $\rightarrow$ `city` ; else `campus`. `category_source` = "source_hint" if a Commons subcategory hint exists, else "heuristic" ; add reason `category_heuristic`.
5. **Warnings:** `low_data` if `len(shown)` < 6 ; `missing_category:<name>` for each of `campus`, `dormitory`, `classroom`, `library`, `city` with zero shown photos; `no_coordinates`, `no_commons_category`, `no_official_site` when applicable; `per-source` `source_unavailable`; `vision_unavailable` if the model was unavailable for the whole run; `time_budget_exceeded` ; `served_from_cache`.
6. `stats.found` = candidates after normalization (Stage C), `per_source` = raw counts before caps, `timings_ms` per stage, `total_ms` at done.

12. HTTP API

All JSON. Errors: `{ "error": { "code": "<snake_case>", "message": "<human text>" } }` with 404 (unknown qid / not a university), 422 (bad params), 503 (pipeline could not even resolve the entity).

Method & path	Params	Response
GET /api/health	—	<code>{ "status": "ok", "version": "<git sha or 'dev'>", "time": "<iso>" }</code>
GET /api/search	<code>q</code> (2–120 chars)	<code>SearchResponse</code> ; result cached 6 h by normalized <code>q</code>
GET /api/profile/{qid}	<code>`refresh=0</code>	<code>1`</code>

GET /api/profile/{qid}/stream	`refresh=0	1`
GET /api/thumb/{id}.jpg	—	JPEG from <code>CACHE_DIR/thumbs</code> , <code>Cache-Control: public, max-age=86400 ; 404 if missing</code>

SSE stream (`sse-starlette EventSourceResponse`; `headers` `Cache-Control: no-cache`, `X-Accel-Buffering: no`; a ping comment every 5 s). Events, each `data` is JSON:

- `header` → `UniversityHeader`
- `description` → `Description`
- `photos` → `{"batch": <int>, "photos": [Photo, ...]}` (may repeat; the client appends)
- `warning` → `Warning`
- `stats` → `Stats`
- `done` → `{"total_ms": <int>, "cached": <bool>}`
- `error` → `{"code": "...", "message": "..."} (terminal, only when nothing could be resolved)`

12.4 Cache (`aiosqlite`, `file {CACHE_DIR}/cache.sqlite`): `tables` `profiles(qid TEXT PRIMARY KEY, json TEXT, created_at TEXT)` and `searches(q_norm TEXT PRIMARY KEY, json TEXT, created_at TEXT)`. TTL: `profiles PROFILE_TTL_HOURS=24`, `searches` 6 h. On a cache hit, verify that the first shown photo's thumbnail file exists on disk; if it doesn't (ephemeral disk after redeploy) → treat as a miss and rebuild. `refresh=1` bypasses and overwrites.

SPA serving: mount `frontend/dist` as static; any GET not starting with `/api` and not matching a static file returns `index.html`.

CORS: allow `ALLOWED_ORIGINS` (default `http://localhost:5173`) in dev; same-origin in production needs nothing.

13. Frontend specification

Routes: `/` search page; `/u/:qid` profile page (`?refresh=1` supported).

Language: `i18n.ts` with `ru` (default) and `en`; toggle in the header; persist in `localStorage("lang")`. All visible strings, reason codes and warning codes go through `i18n`.

Search page

- Large input + button. On submit → `GET /api/search?q=`. Exactly one candidate → navigate to `/u/:qid`. Several → render `CandidateList` (label, description, city, country). Zero → `NotFoundState`: "Университет не найден. Проверьте написание или введите официальное английское название." If `corrected_query` is present show "Показаны результаты по исправленному названию: ...".
- Six example chips (Nazarbayev University, Al-Farabi Kazakh National University, Massachusetts Institute of Technology, ETH Zurich, University of Tokyo, Kostanay Regional University).
- Footer honesty note: "Источники: Wikimedia Commons, Flickr, официальные сайты, Wikipedia. Для небольших вузов данных может быть мало — сервис сообщит об этом явно."

Profile page

- On mount open `EventSource("/api/profile/{qid}/stream")`. If the connection errors before the `header` event → fall back to `GET /api/profile/{qid}` (blocking) and render once. Close the `EventSource` on `done` or `error`.
- `ProgressBar`: four steps with the exact case wording Поиск → Проверка → Категории → Профиль, plus a live elapsed-seconds counter; stays visible until `done`.
- `ProfileHeader`: `name`, local name, city + country, links (official site, Wikipedia), "X км до центра города", cached badge "Результат из кэша, собран N мин назад" with a "Обновить" button (navigates with `?refresh=1`).
- `DescriptionCard`: text in the current language, then "Источники:" links, then the basis note (e.g. "Описание построено по Wikipedia и официальному сайту" / "...только по Wikipedia" / "Недостаточно данных").
- `WarningsBanner`: one line per warning, honest copy (`i18n`), e.g. `low_data` → "Недостаточно проверенных фотографий для этого университета. Показываем только то, что удалось подтвердить."; `source_unavailable` → "Источник {name} недоступен, профиль построен без него."; `time_budget_exceeded` → "Проверка части фото не завершилась в отведённые 30 секунд; такие фото помечены как непроверенные."; `served_from_cache` → see header badge.
- `StatsBar`: "Нашли {found} → убрали {duplicates_removed} дублей и {irrelevant_removed} нерелевантных → показываем {shown} · {total_ms/1000} с".
- `CategoryTabs`: Все · Кампус · Общежития · Аудитории · Библиотеки · Лаборатории · Спорт · Студенческая жизнь · Город, each with a count; tabs with zero shown photos are still rendered and show `EmptyState` "Не удалось найти проверенные фото этой категории."

- `FilterChips` (the four required by the case, exact labels): **Общежитие · Спорт · Лаборатории · Студенческая жизнь** — toggling a chip switches to that category (chips and tabs are two entry points to the same filter state). Plus a switch **"Показать непроверенные ({hidden_unverified})"**.
- `PhotoGrid`: CSS grid, 2 columns on mobile, 3–4 on desktop; `<img loading="lazy" src={thumb_url} alt={title || category}>`; skeleton cards while streaming.
- `PhotoCard`: thumbnail; category chip; `ConfidenceBadge` — verified: green "Подтверждено", likely: amber "Вероятно", unverified: grey "Не подтверждено"; hovering/tapping the badge opens a popover listing the reasons as sentences (i18n of `ReasonCode`, e.g. `commons_category_source` → "Фото из категории университета на Wikimedia Commons", `geo_within_300m` → "Геометка в 300 м от кампуса", `name_on_sign` → "Название университета читается на вывеске", `vision_consistent` → "Визуальная проверка: сцена соответствует категории", `web_search_source` → "Найдено веб-поиском — источник не подтверждён", `city_category_source` → "Фото города из категории города на Wikimedia Commons"); source line: `source_label` + external-link icon → `source_page_url` (`target="_blank" rel="noopener noreferrer"`); date line: "Снято 2019-05-04" / "Опубликовано ..." / "Загружено ..." / "Дата неизвестна"; small author + license line when present.
- `Lightbox`: larger thumbnail, all metadata, the source link, and "Открыть оригинал" (`image_url`).
- `MapCard` (react-leaflet): OSM tiles `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png` with the required attribution "© OpenStreetMap contributors"; markers for the university and the city center; a polyline between them; caption with the distance. Fix the Leaflet default marker icon issue under Vite by importing `leaflet/dist/images/marker-icon.png`, `marker-icon-2x.png`, `marker-shadow.png` and calling `L.Icon.Default.mergeOptions(). Import leaflet/dist/leaflet.css`. Hidden if no coordinates.
- `ErrorState`: network/terminal error with a retry button.

Design: light theme, one accent color, system font stack, generous whitespace, visual hierarchy: header → description → warnings → stats → tabs/filters → grid → map. No decorative animations beyond skeletons. Mobile-first. Keyboard-focusable controls.

14. Build phases and gates (do them in order; commit after each)

Phase 0 — Scaffold. Repo layout, `requirements.txt`, `pyproject.toml` (ruff, pytest asyncio mode auto), `config.py` with all env vars (Section 16.2), logging, `/api/health`, Vite app with a placeholder page, `Dockerfile` (Section 16.1), `docker-compose.yml`, `.env.example`, `.gitignore`, `DECISIONS.md`. Gate: `docker build .` succeeds; `docker run -p 8000:8000 → curl localhost:8000/api/health` returns ok; `npm run build` succeeds.

Phase 1 — Resolver. Section 7.1, 7.2, 7.8(b). `GET /api/search`. Verify the QID label lists (Section 7.1). Save two real responses into `tests/fixtures/` and write `test_wikidata_parse.py`. Gate: `q=Nazarbayev University` → 1 candidate; `q=Columbia` → several with descriptions; `q=Massachusetts Institut of Technology` → MIT via fallback with `corrected_query`; `q=Костанайский региональный университет` → found; `q=asdkjhqwe` → empty, no exception. Header for MIT includes coords, city "Cambridge", official website, commons category.

Phase 2 — Sources + processing (no AI yet). Sections 7.3, 7.4, 8, 11.4 (heuristic categories), 12 (`/api/profile/{qid}` blocking, thumbs endpoint). Vision disabled → every photo scored without vision (max likely). Gate: `scripts/eval.py --no-vision` on MIT and Nazarbayev University: ≥ 15 candidates each, `duplicates_removed` ≥ 1 for MIT, every photo has `source_page_url` and a thumb that loads; parse tests for Commons and Flickr fixtures pass.

Phase 3 — Vision + scoring + description. Sections 9, 10, 7.8(a). Unit tests for all worked examples in Section 10 and for JSON-parse recovery. Gate: MIT cold run ≤ 25 s total; ≥ 90% of vision batches parse on the first attempt (log it); labels distribution sane (some verified, some likely); `ANTHROPIC_API_KEY=invalid` → profile still returns with `vision_unavailable` warnings.

Phase 4 — Streaming + frontend core. Section 12 SSE, Section 13 search + profile pages (without map/official site). Gate: in the browser, header appears ≤ 3 s, first photos ≤ 10 s, `done` ≤ 27 s on a cold university; the four filter chips and the unverified toggle work; every card shows source link, date line, badge with reasons.

Phase 5 — Official site, city, map, optional web search. Sections 7.5, 7.3(e), 7.6, 13 MapCard. Gate: a university with `P856` produces `official_site` photos or the `source_unavailable` warning; city tab has photos for MIT/ETH; map renders with distance.

Phase 6 — Hardening. Cache (12.4), deadline monitor (6), concurrency guard (6.2), `DISABLE_SOURCES` env, error states, i18n completeness, `test_pipeline_degraded.py` (all external HTTP mocked to fail with `resp_x` → profile with warnings, no exception). Gate: cached load < 1 s; `DISABLE_SOURCES=flickr,commons_category` → profile still builds; `HARD_DEADLINE_S=8` → `time_budget_exceeded` appears and the response still completes.

Phase 7 — Evaluation & tuning. Run `scripts/eval.py` on `scripts/universities.txt` with `refresh=1`; write `EVAL.md` (table: university, total_ms, found, shown, verified, likely, hidden, warnings). Manually inspect 5 random verified photos per big university; if a wrong photo is verified, fix the cause (filter/weight), never the symptom. Gate: no university > 30 s; big universities ≥ 15 shown; small university shows `low_data` honestly instead of junk.

Phase 8 — Documentation. Section 17. Screenshots in `docs/`. Final `.env.example`. Final commit to `main`.

If a session's context gets long, start a new session with: "Read SPEC.md and DECISIONS.md, then continue from Phase N."

15. Testing and evaluation

15.1 Unit tests (pytest): scoring worked examples; dedup on synthetic images (generate a 600×400 gradient with Pillow, then a re-encoded copy, a 5%-

cropped copy, and a different image → expect 1 group of 3 + 1 single); textmatch normalization (diacritics, Cyrillic, punctuation); parsers on fixtures; vision JSON recovery (fenced JSON, wrong length); degraded pipeline.

15.2 scripts/eval.py : args --base-url (default http://localhost:8000), --file scripts/universities.txt , --no-vision (sets the env for a local in-process run), --refresh . For each line: search → take the first candidate → GET /api/profile/{qid}?refresh=1 → print a markdown row → write EVAL.md .

15.3 scripts/universities.txt :

```
Nazarbayev University
Al-Farabi Kazakh National University
Kazakh-British Technical University
Kostanay Regional University
Massachusetts Institute of Technology
ETH Zurich
Lomonosov Moscow State University
University of Tokyo
Columbia
Technical University
Massachusetts Institut of Technology
```

15.4 Manual QA checklist (mirrors the jury scenario) — put in README:

- 1. Enter three universities, one of them absent from the demo → each shows a useful profile ≤ 30 s (watch the elapsed counter).
- 2. Enter a misspelling → suggestions or the corrected-name notice; enter nonsense → clear not-found state.
- 3. Open 5 random photos' source links → each opens the real page containing the image.
- 4. Check a category tab with no data → honest empty state; check the unverified toggle.
- 5. Set DISABLE_SOURCES=flickr (or block the host) → profile builds with the warning.
- 6. Reload the same university → cached badge, < 1 s; press "Обновить" → rebuilt.

16. Deployment

16.1 Dockerfile (multi-stage):

```
FROM node:20-alpine AS web
WORKDIR /web
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ ./
RUN npm run build

FROM python:3.11-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY backend/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY backend/ ./
COPY --from=web /web/dist ./app/static
RUN mkdir -p /app/data/thumbs
EXPOSE 8000
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000} --workers 1"]
```

--workers 1 is deliberate (in-memory semaphores and caches). Serve app/static as the SPA.

16.2 Environment variables (.env.example with comments):

Variable	Required	Default	Purpose
ANTHROPIC_API_KEY	for vision/description	—	without it the app runs with vision_unavailable
VISION_MODEL	no	claude-haiku-4-5-20251001	vision batches
TEXT_MODEL	no	= VISION_MODEL	description, spelling
FLICKR_API_KEY	no	—	Flickr source
GOOGLE_CSE_KEY , GOOGLE_CSE_CX	no	—	optional web-search booster

CONTACT_EMAIL	yes	team@example.com	User-Agent contact (Wikimedia policy)
CACHE_DIR	no	./data	sqlite + thumbs
PROFILE_TTL_HOURS	no	24	
HARD_DEADLINE_S	no	27	
VISION_BATCH_SIZE / VISION_CONCURRENCY	no	8 / 4	
DOWNLOAD_CONCURRENCY	no	16	
MAX_CANDIDATES	no	120	
DISABLE_SOURCES	no	``	comma list of source names for testing
ALLOWED_ORIGINS	no	http://localhost:5173	dev CORS
LOG_LEVEL	no	INFO	
PORT	no	8000	

16.3 Hosting notes (put in README): deploy the single container to Railway, Fly.io or Render. Free tiers that sleep add 30–60 s cold start, which destroys the speed criterion — either use a plan that does not sleep or configure an external pinger (e.g. cron-job.org) hitting `/api/health` every 5 minutes. The disk is ephemeral: the cache self-heals (12.4).

16.4 Local dev: `docker compose up` (backend on 8000 with `./data` mounted) and `npm run dev` in frontend (proxy `/api` → 8000).

17. Documentation (Russian; code comments and this spec in English)

README.md sections, in this order and with these exact headings (they mirror the case's submission requirements):

- Задача** — one paragraph, the case in our words.
- Решение** — what the user gets, the honesty principle, screenshot.
- Стек**
- Архитектура** — text diagram of the pipeline (Stages A–I), one paragraph per stage, where the 30-second budget lives.
- Инструкция запуска** — Docker, local dev, all env vars, how to get each key.
- Тестовый сценарий** — Section 15.4 verbatim, with the expected result of each step.
- Роли команды** — placeholders `<Имя – роль>` for three people (the user fills in).
- Источники данных** — table: source, what we take, how the source link is formed, date field, licence handling, known limitations.
- AI / API** — models and exactly what each does; link to `backend/app/ai/prompts.py`; note that the vision model never verifies on its own.
- Готовые компоненты** — every third-party library and template used, with one line on what it does (disclosure requirement).
- Ограничения** — honest list: coverage depends on Commons/Flickr/official sites; JS-only sites; Flickr geotag noise; possible stock images on official sites; vision misclassification; quota limits; cold starts.
- Методы проверки** — the Section 10 table and thresholds, in Russian.
- Развитие** — what we would add next (new sources as plugins in `sources/`, comparison of two universities, reviews).

TECHNICAL_NOTES.md: models, APIs, libraries, external services, data flow, verification methods, evaluation results (copy of `EVAL.md`), and the statement that all code was written during the hackathon window with AI coding assistants.

DECISIONS.md: one line per decision, chronological.

18. Definition of Done (all must be true)

- `docker build + run` works from a clean clone with only `.env` provided.
- Cold profile for each of the 8 real universities in 15.3 completes ≤ 30 s; header ≤ 3 s; first photos ≤ 10 s.
- Every shown photo has: clickable source, date or "Дата неизвестна", category, badge, reasons popover, author/licence when known.

- Duplicates and near-duplicates are removed; `StatsBar` shows the counts.
 - Ambiguous name → candidate list; misspelled → correction; nonsense → not-found state; small university → `low_data` banner; disabled source → warning; invalid AI key → `vision_unavailable`, still a profile.
 - The four required filter chips and all category tabs work; unverified photos hidden by default.
 - Map with distance to the city center when coordinates exist.
 - Description with source links and basis note, in RU and EN.
 - Cached results labeled and refreshable.
 - No secrets in git; `.env.example` complete; all unit tests green; `ruff` clean.
 - README (RU) with all 13 sections, `TECHNICAL_NOTES.md`, `EVAL.md`, `DECISIONS.md` present.
 - No hardcoded universities, photos, timings or confidence values anywhere in the code.
-

19. Explicit non-goals (do not build)

Comparison of two universities; climate/transport/cost-of-living data; student reviews; user accounts; a database of universities; CLIP or any locally hosted ML model; server-side rendering; PWA/offline; multi-worker deployment; admin panels; analytics.

20. Known pitfalls checklist (read before Phases 2–5)

- Wikimedia APIs return 403 without a descriptive `User-Agent`; `upload.wikimedia.org` returns 429 under high parallelism → the 8-connection semaphore.
- Flickr geo search without `min_taken_date` (a "limiting agent") returns an error or nothing.
- Commons `DateTimeOriginal` is free text; never trust it without the regex; fall back to `upload_timestamp` labeled "uploaded".
- Official sites often block hotlinking → `<img>` always points to `/api/thumb/...`, never to the original.
- Proxies buffer SSE → `X-Accel-Buffering: no` and periodic pings.
- Pillow decompression bombs → `MAX_IMAGE_PIXELS` and the 4 MB cap; decode in threads to keep the event loop free.
- Anthropic rate limits under jury load → batching + `VISION_CONCURRENCY`; on 429 back off, never fail the profile.
- Leaflet default marker icons are missing under Vite unless imported explicitly.
- `EventSource` cannot send headers → all stream parameters are query params.
- `datetime.now()` must be timezone-aware UTC everywhere (`datetime.now(timezone.utc)`).
- `asyncio.gather(..., return_exceptions=True)` for source fan-out, so one failing source never cancels the others.